

Esercitazione: Costruisci un'App per Monitorare il Budget Personale

Obiettivi dell'esercitazione: In questa esercitazione, svilupperai un'applicazione web in PHP 8 utilizzando il pattern MVC, la programmazione orientata agli oggetti (OOP), namespace, file upload, sessioni, REST API, MySQL, e PDO. L'app permetterà agli utenti di monitorare il proprio budget personale, registrando entrate e uscite, categorizzando le spese, e visualizzando riepiloghi e statistiche finanziarie.

Funzionalità richieste:

1. Registrazione e Login:

- Gli utenti devono poter registrarsi e fare login per gestire le proprie finanze in modo sicuro.
- Le credenziali devono essere protette utilizzando una password hashata.

2. Gestione delle Sessioni:

- Gestisci le sessioni per mantenere gli utenti autenticati e proteggere i dati personali.

3. Monitoraggio delle Entrate e Uscite:

- Gli utenti devono poter aggiungere, modificare e visualizzare entrate e spese.
- Ogni transazione deve contenere: nome, data, importo, categoria (ad esempio "Affitto", "Stipendio", "Spese Generali").

4. File Upload per Ricevute:

- Gli utenti devono poter caricare immagini o PDF delle ricevute o fatture delle spese.

5. REST API:

- Crea una REST API che permetta di:
 - Recuperare l'elenco delle transazioni in formato JSON.
 - Aggiungere, modificare e cancellare le transazioni tramite richieste HTTP.

6. Dashboard Finanziaria:

- Implementa una dashboard che mostri statistiche finanziarie, come il bilancio mensile e il totale delle spese per categoria.

7. Gestione Utenti:

- Crea un modulo per modificare le informazioni del profilo utente (nome, email, obiettivi finanziari, ecc.).

Requisiti Tecnici:

- **PHP 8:** Utilizza le funzionalità di PHP 8, come le *union types* e la gestione delle eccezioni.

- **MVC (Model-View-Controller):** Organizza il progetto secondo il pattern MVC per separare la logica di business (Model), il flusso delle richieste (Controller) e la parte grafica (View).
 - **OOP e Namespace:** Organizza il codice in classi e utilizza i namespace per gestire la logica relativa agli utenti, alle transazioni e alla gestione delle sessioni.
 - **File Upload:** Implementa la funzionalità per il caricamento delle ricevute. I file devono essere memorizzati in una directory del server e il percorso salvato nel database.
 - **REST API:** Implementa una REST API per la gestione delle operazioni CRUD (Create, Read, Update, Delete) sulle transazioni, con dati in formato JSON.
 - **Sessioni:** Usa sessioni per gestire l'autenticazione dell'utente e proteggere i dati sensibili.
 - **MySQL e PDO:** Utilizza PDO per interagire con il database MySQL, usando prepared statements per prevenire attacchi SQL injection.
-

Struttura del Progetto

1. Directory Principale del Progetto:

- **/app** — Contiene i modelli, i controller e le viste.
- **/public** — Contiene file accessibili pubblicamente come **index.php**, CSS, immagini, e file caricati.
- **/config** — Configurazione del database e altre impostazioni generali.
- **/vendor** — Dipendenze gestite da Composer.

2. Modelli OOP:

- **User:** Gestisce la registrazione, login e gestione del profilo utente.
- **Transaction:** Gestisce l'aggiunta, modifica, cancellazione e recupero delle transazioni finanziarie.
- **Session:** Gestisce l'autenticazione e la gestione delle sessioni utente.

3. Controller:

- **UserController:** Per la gestione di utenti e autenticazione.
- **TransactionController:** Per la gestione delle transazioni.
- **ApiController:** Per la gestione delle API REST per le transazioni.

4. Views (HTML e PHP):

- **User Views:** Pagine per la registrazione, il login e la gestione del profilo.
 - **Transaction Views:** Pagine per la visualizzazione e gestione delle transazioni e della dashboard finanziaria.
-

Database MySQL

Tabella **users**:

```
CREATE TABLE `users` (  
  `id` INT AUTO_INCREMENT PRIMARY KEY,  
  `name` VARCHAR(100) NOT NULL,  
  `email` VARCHAR(100) NOT NULL UNIQUE,  
  `password` VARCHAR(255) NOT NULL,  
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Tabella transactions:

```
CREATE TABLE `transactions` (  
  `id` INT AUTO_INCREMENT PRIMARY KEY,  
  `user_id` INT NOT NULL,  
  `description` VARCHAR(255) NOT NULL,  
  `date` DATE NOT NULL,  
  `amount` DECIMAL(10,2) NOT NULL,  
  `category` ENUM('Affitto', 'Stipendio', 'Spese Generali', 'Altro') NOT NULL,  
  `receipt` VARCHAR(255),  
  `created_at` TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (`user_id`) REFERENCES `users`(`id`) ON DELETE CASCADE  
);
```

Dati Fake per users:

```
INSERT INTO `users` (`name`, `email`, `password`)  
VALUES  
( 'Giovanni Rossi', 'giovanni@example.com', '$2y$10$abc123...'),  
( 'Maria Bianchi', 'maria@example.com', '$2y$10$def456...'),  
( 'Luca Verdi', 'luca@example.com', '$2y$10$ghi789...');
```

Dati Fake per transactions:

```
INSERT INTO `transactions` (`user_id`, `description`, `date`, `amount`,  
  `category`, `receipt`)  
VALUES  
(1, 'Affitto Settembre', '2023-09-01', 850.00, 'Affitto', 'uploads/affitto.jpg'),  
(2, 'Stipendio', '2023-09-05', 1500.00, 'Stipendio', NULL),  
(3, 'Spesa al supermercato', '2023-09-10', 80.50, 'Spese Generali',  
  'uploads/scontrino.jpg');
```

Passaggi dell'Esercitazione:

1. Imposta il Database:

- Crea il database MySQL e inserisci le tabelle e i dati forniti.

2. Configurazione del Progetto:

- Configura l'ambiente di sviluppo, inclusa la configurazione del database e l'autoloading delle dipendenze tramite Composer.

3. Implementazione del Modello (Model):

- Crea le classi **User** e **Transaction** che interagiscono con il database tramite PDO, permettendo la gestione degli utenti e delle transazioni.

4. Creazione del Controller:

- Crea **UserController** per gestire la registrazione e il login, e **TransactionController** per la gestione delle transazioni (aggiunta, modifica, cancellazione).

5. Gestione delle Viste (View):

- Crea le pagine HTML per la gestione degli utenti (login/registrazione) e la visualizzazione delle transazioni.

6. File Upload:

- Implementa la funzionalità di caricamento delle ricevute delle spese (immagini o PDF) e memorizza il percorso dei file nel database.

7. REST API:

- Crea una REST API per permettere l'aggiunta, la modifica, la cancellazione e la visualizzazione delle transazioni in formato JSON.

8. Sessioni e Sicurezza:

- Gestisci le sessioni per mantenere gli utenti autenticati. Proteggi le pagine sensibili per evitare accessi non autorizzati.

Conclusione:

Al termine di questa esercitazione, avrai sviluppato un'applicazione completa per monitorare il budget personale, permettendo agli utenti di gestire le proprie finanze, visualizzare statistiche e utilizzare una REST API per la gestione delle transazioni.